Module-V

1. Explain overview of Operating System.

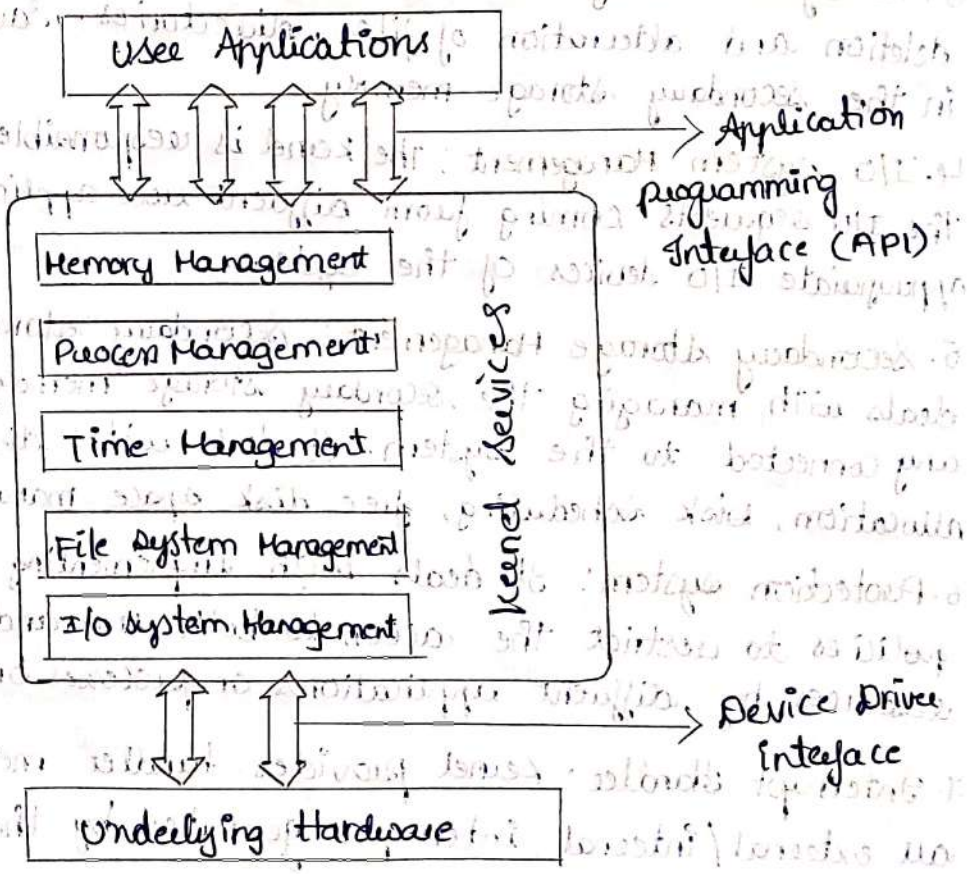
Operating System:-

The operating system acts as a bridge between the user applications / tasks and the underlying system resources through a set of system functionalities and services.

The primary functions of an operating system is

→ Make the system convenient to use

→ organize and manage the system resources efficiently and correctly.



Architecture of operating system.

The kernel:

The kernel is the core of the operating system. It is responsible for managing the system resources and the communication among the hardware and other system services.

Scanned with OKEN Scanner

Kernel acts as the abstraction layer between system resources and user applications. Kernel contains a set of system libraries and services.

For a general purpose OS, the kernel contains different services like.

1. Process Management - It deals with managing the processes/tasks. It includes setting up the memory space for the process, loading the process's code, setting up and managing the process control Block (PCB) etc.

2. Primary Memory Management - The Memory Management unit of the kernel are responsible for keeping track of which part of memory area is currently used by which process., Dynamic memory allocation.

3. File System Management : It is responsible for the creation, deletion and alteration of files, directories. Saving of files in the secondary storage memory.

4. I/O System Management : The kernel is responsible for routing the I/O requests coming from different user applications to the appropriate I/O devices of the system.

5. Secondary Storage Management : Secondary storage management deals with managing the secondary storage memory devices, if any connected to the system. It deals with disk storage allocation, Disk Scheduling, free disk space management.

6. Protection System : It deals with implementing the security policies to restrict the access to both user and system resources by different applications or processes or users.

7. Interrupt Handler : Kernel provides handler mechanism for all external/internal interrupts generated by the system.

Kernel Space and User Space :-

The memory space at which the kernel code is located is known as 'Kernel Space'.

All user applications are loaded to a specific area of primary memory and this memory area is verified as

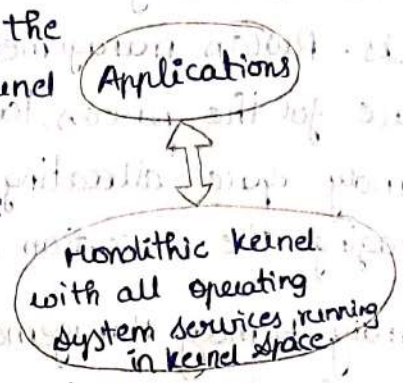
user space.

The partitioning of memory into kernel and user space is purely operating system dependent.

Monolithic kernel.

→ All kernel services run in the kernel space

→ All kernel modules run within the same memory space under single kernel thread

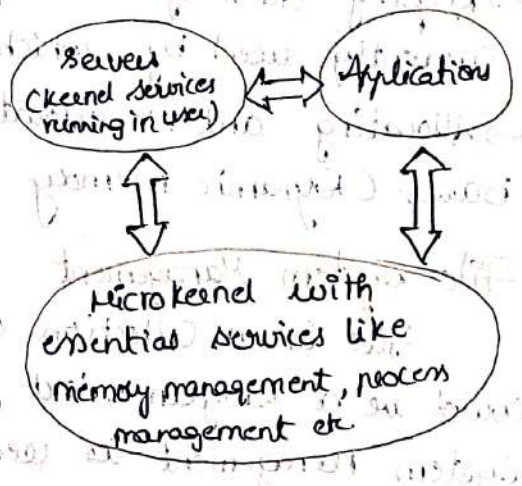


→ The major drawback of monolithic kernel is that any error or failure in any one of the kernel modules leads to the crashing of the entire kernel application.

→ Ex :- LINUX, SOLARIS, MS-DOS kernels.

Microkernel :

The microkernel design incorporates only the essential set of operating system services into the kernel



Rest of the OS services are implemented in programs known as servers

The kernel design is highly modular produces 'OS-neutral' abstraction

Ex: QNX, Minix 3 kernels.

2. Discuss Functions of operating systems.

Functions of operating systems :-

Functions of operating systems are as follows.

↳ Task Management (Multitasking):

• Scheduling : The OS manages which tasks run at

Scanned with OKEN Scanner

what times, ensuring time-sensitive tasks execute promptly. In RTOS, this is especially critical to ensure real time performance.

Process Management:

Process Management deals with managing the processes/tasks. Process management includes setting up the memory space for the process, loading the process's code into the memory space, allocating system resources, scheduling and managing the execution of the process.

Primary Memory Management:

The term primary memory refers to the volatile memory (RAM) where processes are loaded and variables and shared data associated with each process are stored. It is responsible for

→ keeping track of which part of the memory area is currently used by which process

→ Allocating and De-allocating memory space on a need basis (Dynamic memory allocation).

File System Management

File is a collection of related information. A file could be a program, text files, image files, etc. The file system management is responsible for.

→ The creation, deletion and alteration of files, directories.

→ Saving of files in the secondary storage memory.

→ providing automatic allocation of file space based on the amount of free space available

→ providing a flexible naming convention for the files.

Device Management: The device management is responsible for, loading and unloading of device drivers

• Exchanging information and the system specific control signals to and from the device.

Scanned with OKEN Scanner

Secondary Storage Management:

It deals with managing the secondary storage memory devices, if any, connected to the system. Secondary memory is used as backup medium for programs and data since the main memory is volatile.

This service deals with

Disk storage allocation

Disk scheduling

Free disk space management.

Protection systems: Most of the modern OS's are designed in such a way to support multiple user with different levels of access permissions. Protection deals with implementing the security policies to restrict the access to both user and system resources by different applications/users.

3. Explain RTOS Task Scheduling.

Real Time Operating System Task Scheduling:

In a multitasking system, there should be some mechanism in place to share the CPU among the different tasks and to decide which process/task it to be executed at a given point of time.

Determining which task/process is to be executed at a given point of time is known as task/process scheduling

The scheduling policies are implemented in an algorithm and it is run by the kernel as a service.

The kernel service/application, which implements the scheduling algorithm is known as 'scheduler'.

The task scheduling policy can be pre-emptive, non-pre emptive or co-operative.

Depending on the scheduling policy the process scheduling decision may take place when a process switches its state

Scanned with OKEN Scanner

to
→ Ready state from Running state
→ Blocked/wait state from Running state
→ Ready state from Blocked/wait state
→ Completed state.

Task Scheduling - Scheduler selection:

The Selection of a scheduling Criteria/ algorithm should consider.

- CPU Utilization: The scheduling algorithm should always make the CPU utilization high. CPU utilization is a direct measure of how much percentage of the CPU is being utilized.
- Throughput: This gives an indication of the number of processes executed per unit of time. The throughput for a good scheduler should always be higher.
- Turnaround Time: It is the amount of time taken by a process for completing its execution. It includes the time spent by the process for waiting for the main memory, time spent in the ready queue, completing the I/O operations and execution. It should be minimum for good scheduling.
- Waiting Time: It is the amount of time spent by a process in the 'Ready' queue waiting to get the CPU time for execution. The waiting time should be minimal for a good scheduling algorithm.
- Response Time: It is the time elapsed between the submission of a process and the first response. For a good scheduling algorithm, the response time should be ~~least~~ as least as possible.

→ A good scheduling algorithm has high CPU utilization, minimum Turn Around Time (TAT), maximum Throughput and least response time.

Scanned with OKEN Scanner

4. Explain keil RTX Thread.

Keil RTX :-

Keil RTX (Real-Time eXtension) is a real-time operating system (RTOS) designed for ARM Cortex-M microcontrollers. It provides multi-threading capabilities that allow an embedded system to run multiple tasks concurrently.

Keil RTX Threads:

Thread : In keil RTX, a thread is a standalone piece of code that executes a specific function. It has its own stack and can run independently of other threads. Each thread has a priority, which determines when it will run in relation to other threads.

Thread States: A Thread in RTX can be in one of several states:

- Ready : Waiting to be scheduled for execution.
- Running : Actively executing on the CPU
- Blocked : Waiting for a resource, such as a semaphore, event or delay.
- Inactive : Not currently scheduled or waiting.

Scheduling: RTX uses a priority-based, preemptive scheduling algorithm. Higher-priority threads are given preference, meaning they can interrupt (preempt) lower-priority threads if needed. If two threads have the same priority, RTX uses round-robin scheduling, allowing each thread equal CPU time.

Time-Slicing : RTX allows for time-slicing among threads of the same priority, which helps in giving all threads a chance to execute even if they have

Scanned with OKEN Scanner

the same priority level.

Thread Management Functions:

RTX provides APIs for creating, deleting, suspending and resuming threads. These functions let you control when and how each thread executes.

Thread Communication:

Threads can communicate and synchronize using various methods, such as message queues, semaphores, and mutexes.

These tools help manage shared resources and prevent conflicts in multi-threaded applications.

5. Discuss the concept of RTOS on Mbed platform.

An RTOS (Real-Time Operating System) is an operating system specially designed for real-time applications, which is crucial for embedded systems.

The Mbed platform, developed by Arm, supports RTOS functionality, particularly through Mbed OS, which includes built-in RTOS support to manage tasks, timers and other concurrency mechanisms.

Key Concepts of RTOS on the Mbed platform:

1. Multithreading:

Mbed OS allows multiple threads to run simultaneously, each performing different tasks. Threads can run independently, and the RTOS scheduler manages which thread gets CPU time based on priority and system requirement.

2. Task Scheduling:

The Mbed RTOS uses a priority-based preemptive scheduling approach. Each thread is assigned a priority, and the scheduler decides which thread to execute based on its priority level.

Higher-priority threads can interrupt lower-priority threads, ensuring timely responses for critical tasks.

Scanned with OKEN Scanner

3. Inter-Thread Communication:

Mbed OS provides several mechanisms for threads to communicate and data, such as:

- Mutexes for preventing concurrent access to shared resources.
- Semaphores for managing resources shared by multiple threads.
- Queues and Event flags to signal and pass message between threads without direct interaction.

4. Timers and Delays:

Mbed OS includes timers to measure time intervals and manage task delays. With Timer and Ticker classes, you can execute time-based functions accurately.

5. Memory Management:

The RTOS on Mbed OS provides dynamic memory management and stack allocation for each thread. Since, embedded systems often have limited memory, careful memory management is essential to avoid fragmentation or memory overflows.

6. Low Power Operations:

Mbed OS includes power management features suitable for battery-operated devices, where the RTOS can put the system in a low-power state when idle, thus extending battery life.

Practical Benefits of using RTOS in Mbed:

- Concurrency: The RTOS allows handling multiple tasks without blocking each other, ideal for complex IoT or robotics projects.
- Timing and Deadlines: Mbed OS RTOS makes it easier to meet timing constraints, which are often critical in embedded applications.
- Modularity: RTOS allows breaking down an application into smaller, manageable threads, making code easier to develop and debug.

Scanned with OKEN Scanner

Using RTOS on embed platform enhances real-time task handling and resource management, making it suitable for embedded systems that demand reliability and timely responses.

6. Briefly explain Mbed RTOS API

The mbed RTOS API, provided by Arm for its mbed platform, is designed to help developers build responsive, multitasking applications for embedded systems. Key features of the mbed RTOS API include:

Thread Management:

The API allows the creation, control, and termination of threads, enabling multitasking in embedded applications. Each thread can execute independently, with its own stack and priority, facilitating the division of tasks across multiple threads for organized and efficient operation.

Developers can adjust thread priorities, allowing critical tasks to be prioritized over non-critical ones, which is essential in RTOS where response time is crucial.

Mutexes and Semaphores:

The mbed RTOS API includes mutexes and semaphores for managing access to shared resources among threads.

A mutex ensures that only one thread can access a resource at a time, preventing conflicts.

Semaphores, on the other hand allows for limited access by a specified number of threads, useful for managing access to resources with limited capacity.

Timers and Delays:

The API provides timers and delay functions to handle precise timing requirements, crucial for scheduling tasks accurately in real-time systems.

Scanned with OKEN Scanner

Timers can be used to trigger tasks or functions at specified intervals, while delay functions pause a thread for a set time, ensuring tasks execute in sync with real-world events.

Event Flags and Queues:

Event flags and message queues enable communication and synchronization between threads.

Even flags are used to signal between threads or tasks, helping coordinate actions based on specific conditions.

Message queues allow data exchange between threads, enabling organized and prioritized message passing.

Memory Management:

The Mbed RTOS API includes functions for efficient memory allocation and deallocation to handle memory-constrained environments typically in embedded systems.

This ensures memory is used effectively, minimizing waste and avoiding fragmentation, which can be a significant issue in embedded device with limited RAM.

The Mbed RTOS API provide essential tools for developing real-time, multitasking applications on embedded systems. With its support for thread management, resource synchronization, timing, communication, and memory handling, it simplifies the creation of robust and efficient embedded applications, particularly in fields requiring precise control and reliability, such as IOT, automation and robotics.

Scanned with OKEN Scanner

7. Difference between mutex and semaphore.

Mutex	Semaphore.
A Mutex (Mutual Exclusion) is a locking mechanism that ensures only one thread can access a resource at a time	A semaphore is a signaling mechanism that controls access to a resource by multiple threads through a counter.
Used to protect shared resources that can be accessed by one thread at a time	used to control access to a pool of resources, allowing multiple threads to use a resource up to a certain limit
Typically initialized in a default state	Initalized with a specific count representing the number of available resources.
A thread that locks a mutex must unlock it; only the owner thread can release the mutex	Any thread can signal a semaphore not necessarily the one that acquire it
Value Range is Binary: can only be locked (1) or unlocked (0).	Can have a non-negative integer value representing the no. of available resources
If a mutex is locked other threads attempting to lock it will be blocked until it is unlocked	If the semaphore count is zero, threads trying to decrement it will be blocked until it becomes positive
Higher risk of deadlock if not managed properly especially with multiple mutexes	Lower risk of deadlock, though possible with incorrect usage

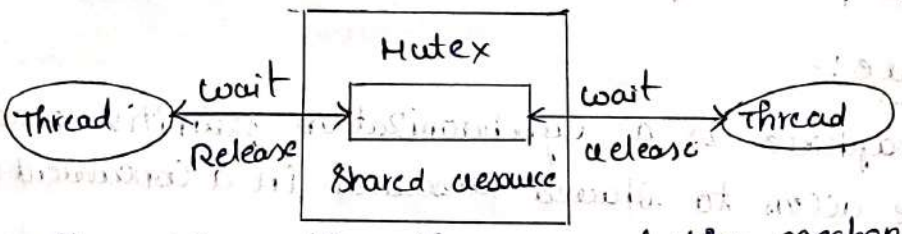
Scanned with OKEN Scanner

Susceptible to priority inversion problems, where a lower-priority thread holds a mutex needed by higher-priority thread.	Priority inversion can also occur but less common compared to mutexes.
Slightly higher overhead due to ownership tracking	Generally lower overhead due to simpler signaling mechanism.
Ensuring exclusive access to a single resource (e.g., a shared variable)	Controlling access to a finite number of identical resources. (e.g., a pool of database connections).

8. Write short notes on Mutex and semaphore.

Mutex:-

A Mutex (Mutual exclusion) is a synchronization primitive used to prevent multiple threads from simultaneously accessing a shared resource, ensuring that only one thread can enter the critical section at a time.



It achieves this through a locking mechanism, making it a key tool for solving critical section problem in concurrent programming.

When a thread needs to access a resource protected by a mutex, it must first acquire the mutex. While the mutex is locked, other threads attempting to acquire it will blocked until the mutex is released. Once the thread finishes its work with the shared resource, it releases the mutex, allowing other threads to access.

Scanned with OKEN Scanner

the resource.

features:-

- Lock Based Mechanism: It operates as a binary semaphore by locking (1) and unlocking (0).
- Ownership: This ensures controlled access and prevents accidental release by other threads.
- Priority: Mutexes are designed to keep higher-priority tasks blocked for the shortest time possible, reducing the risk of priority inversion & deadlock scenarios.
- Process Synchronization: By using a mutex, we achieve process synchronization, allowing threads to work with shared resources without interfacing with each other.

Advantages:-

→ Data consistency
→ Simplicity
→ Efficient performance

Disadvantages:-

→ Potential CPU wastage
→ Single Thread Access
→ Risk of starvation
→ Deadlock possibility.

Semaphore:-

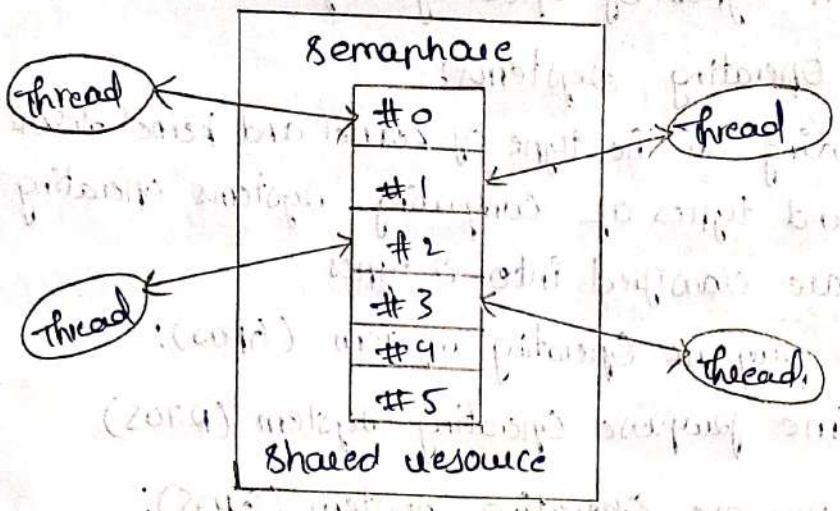
A semaphore is a synchronization primitive used to manage access to shared resources in a concurrent system.

Semaphores are used not only as process synchronization tools but also as signaling mechanism to indicate whether a resource is being acquired or released.

There are two types of semaphores

→ Binary semaphores
→ Counting semaphores

Scanned with OKEN Scanner



Binary Semaphore :-

It has only 2 values 0 or 1. It is often used to achieve mutual exclusion, similar to mutex lock. These are used for process synchronization where only one process can access a shared resource at a time.

Counting Semaphore :-

A counting semaphore can have a range of values, typically starting from 0 upto a maximum value, which represents the no. of available resources. These are ideal for managing access to pool of identical resources.

Features :-

- Initilization - It is initialized with the number of available resources
- Access Control - It restricts or grants access to resources based on its value. when thread acquires a resource value decreased if not increases.
- ~~Autonomal~~ Atomic operations - It relay on two key atomic signal operations wait() & signal().

Advantages :-

→ Multiple thread Access
→ Machine - independent
→ Flexible resource management
→ NO Busy waiting in proper implementation
→ Modularity in Microkernel

Disadvantages :-

→ Complexity
→ Deadlock risk
→ loss of modularity
→ priority inversion
→ Not ideal for Mutual Exclusion.

Scanned with OKEN Scanner

9. Explain types of Operating systems.

Types of operating systems:-

Depending on the type of kernel and kernel services, purpose and types of computing systems operating systems are classified into 2 types

1. General purpose Operating system (GPOS):
2. Real Time purpose Operating system (RTOS)

General purpose Operating system (GPOS):

→ Operating systems, which are deployed in general computing systems.

→ The kernel is more generalized and contains all the required services to execute generic applications.

→ May inject random delays into application software and thus cause slow responsiveness of an application at unexpected times.

→ Usually deployed in computing systems where deterministic behavior is not an important criterion.

→ personal Computer / Desktop system is a typical example for a system where GPOS are deployed.

→ Windows XP/MS-DOS etc are examples of General purpose operating system.

Real Time purpose Operating system (RTOS):

Operating systems, which are deployed in embedded system demanding real-time response

Deterministic in execution behavior. Consumes only known amount of time for kernel applications.

Implements scheduling policies for executing the highest priority task/application always.

Implements policies and rules concerning time-critical allocation of a system's resources.

Scanned with OKEN Scanner

→ Windows CE, QNX, Vx Works, Micro C/OS-II etc are examples of RTOS.

The Real Time Kernel:

The kernel of Real Time operating system is referred as Real Time kernel. In complement to the conventional OS kernel, the Real Time kernel is highly specialized and it contains only the minimal set of services required for running the user applications/tasks.

The basic functions of Real Time kernel are

1. Task/process Management: This involves creating, deleting and controlling tasks. The kernel manages the states and context of each task, ensuring efficient switching between them.

2. Task/process scheduling: This is the core of a real-time system, where the kernel decides the order in which tasks execute based on their priorities and deadlines. It ensures timely execution, particularly for high-priority tasks.

3. Task/process Synchronization: Since multiple tasks may need to access shared resources, the kernel provides mechanisms to prevent conflicts and ensure that resources are accessed in coordinated manner.

4. Error/Exception Handling: Real-time systems often have critical tasks, so the kernel includes mechanisms to detect and handle errors/exceptions, ensuring that the system remains stable or recovers gracefully.

5. Memory Management: The kernel allocates and deallocates memory to tasks as needed, ensuring efficient usage and preventing memory leaks. Some real-time kernels use specialized memory management to minimize delays.

6. Interrupt handling: Interrupt allow the kernel to respond quickly to external events. The kernel prioritizes

Scanned with OKEN Scanner

and handles these interruptions efficiently, resuming tasks with minimal delay.

Time Management: This involves managing times and clocks; essential in real-time systems for tracking deadlines and time intervals, ensuring tasks are completed within specified time constraints.

10. Write overview of Real-Time Operating System.

Refer to (9).

11. Explain difference between General operating system and Real time operating system.

General operating system	Real Time operating system
It is used for general universal applications	It is used for dedicated electronics applications.
Task deadlines are not present	Task deadlines are present.
Time response of OS is not deterministic	Time response of OS is deterministic
Kernel is non-preemptive or has long non-preemptive code sections.	Kernel is preemptive either completely or up to maximum degree
Behaviour of GPOS is not predictable	Behaviour of RTOS is predictable
Depending on application we cannot customise GPOS (Not scalable)	Depending on application we can customise RTOS (Scalable).

Scanned with OKEN Scanner

It does not optimises the memory resources.	It optimises the memory resources
High interrupt latency	Low interrupt latency.
It is normally stored in hard disk	It is normally stored in semiconductor memory like EEPROM, Flash EEPROM
Applications are compiled and linked separately from the OS	Applications usually linked with RTOS
GPOS examples are Linux, windows and IOS etc	RTOS examples are Contiki source code, Free RTOS etc.
GPOS is designed to multi user environment	RTOS is designed to single user environment
Applications: personal computers, laptops, Tablets.	Applications: Medical imaging system, Industrial control systems, ATM's, etc.

12. Explain kernel and types of kernel in Linux

Refer from ①

Scanned with OKEN Scanner